

Build a complete web application called Rebl. Build everything listed here in order without stopping.

=== TECH STACK ===

React + Vite | TailwindCSS | React Router v6 | Supabase | Gemini API (FREE) | lucide-react | react-hot-toast | recharts | canvas-confetti

Install: npm install react-router-dom @supabase/supabase-js tailwindcss postcss autoprefixer lucide-react react-hot-toast recharts canvas-confetti

Tailwind brand colors in tailwind.config.js extend.colors:

dark: '#0F0F1A', accent: '#E63946', cream: '#F1FAEE',
muted: '#8D99AE', gold: '#FFB703', card: '#16162A'

=== .env FILE ===

VITE_SUPABASE_URL=

VITE_SUPABASE_ANON_KEY=

VITE_GEMINI_API_KEY=

=== FILE: src/lib/supabase.js ===

```
import { createClient } from '@supabase/supabase-js';  
  
export const supabase = createClient(import.meta.env.VITE_SUPABASE_URL,  
import.meta.env.VITE_SUPABASE_ANON_KEY);
```

=== FILE: src/lib/gemini.js (ALL AI CALLS GO THROUGH THIS) ===

```
export async function callAI(prompt, systemPrompt = '') {  
  const fullPrompt = systemPrompt ? systemPrompt + '\n\n' + prompt : prompt;  
  const res = await fetch(  

```

```

`https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-
flash:generateContent?key=${import.meta.env.VITE_GEMINI_API_KEY}`,

  { method:'POST', headers: {'Content-Type':'application/json'},

    body: JSON.stringify({ contents:[{parts:[{text:fullPrompt}]}]},
generationConfig:{temperature:0.8, maxOutputTokens:800} }) }

);

const data = await res.json();

if (data.error) throw new Error(data.error.message);

return data.candidates[0].content.parts[0].text;

}

export async function callAIJSON(prompt, systemPrompt = "") {

  const text = await callAI(prompt + '\n\nReturn ONLY valid JSON. No markdown. No
backticks.', systemPrompt);

  return JSON.parse(text.replace(/``json|``/g,"").trim());

}

```

=== FILE: src/App.jsx ===

Set up React Router v6 with these routes:

```

/ -> LandingPage

/login -> AuthPage mode="login"

/signup -> AuthPage mode="signup"

/brand/signup -> BrandSignup

/dashboard -> ProtectedRoute wrapping Dashboard

/profile/:username -> CollectorProfile

/vault/:username -> ProtectedRoute wrapping Vault

/add-item -> ProtectedRoute wrapping AddItem

/tribe -> ProtectedRoute wrapping TribePage

/post-purchase/:id -> PostPurchase

/brand/:slug -> BrandPage

```

/brand-dashboard -> BrandRoute wrapping BrandDashboard

/s/:brandSlug -> BrandSubdomainPage

ProtectedRoute: check supabase.auth.getSession(), redirect /login if no session.

BrandRoute: same check + fetch profile.role, redirect /login if role !== 'brand'.

Add Toaster from react-hot-toast at root.

=== DESIGN SYSTEM (use throughout all pages) ===

Background: bg-[#0F0F1A]

Cards: bg-[#16162A] border border-[#2A2A3E] rounded-lg

Primary button: bg-[#E63946] hover:opacity-90 text-white font-bold px-6 py-3 rounded-lg

Ghost button: border border-[#2A2A3E] text-[#F1FAEE] hover:border-[#E63946] px-6 py-3 rounded-lg

Input: bg-[#0F0F1A] border border-[#2A2A3E] focus:border-[#E63946] text-[#F1FAEE] px-4 py-3 rounded-lg outline-none w-full

Text: text-[#F1FAEE] primary, text-[#8D99AE] muted, text-[#E63946] accent, text-[#FFB703] gold

Always mobile-first. Dark everywhere.

=== PAGE: src/pages/LandingPage.jsx ===

NAVBAR (sticky, dark, border-b border-[#2A2A3E]):

Left: "Rebl" font-bold text-3xl text-[#F1FAEE]

Right: Login ghost button small + "Join Free" red button small

HERO (min-h-screen flex items-center dark bg):

Animated background: absolute inset-0 with CSS radial-gradient animation (subtle purple glow top-right)

Headline: "Mass market is over." text-6xl md:text-8xl font-bold text-[#F1FAEE]

Sub: "Rebl is for collectors who refuse to blend in. Showcase what you own. Prove it's real. Find your people." text-xl text-[#8D99AE] max-w-2xl mt-4

Buttons row: "Start Your Collection" red large + "Explore Drops" ghost large, mt-8

3 stat cards row mt-12: "500+ Collectors" | "20+ Brands" | "1000+ Items"

Each stat card: bg-[#16162A] border border-[#2A2A3E] rounded-lg px-6 py-4, number text-2xl font-bold text-[#E63946]

THE PROBLEM (py-24):

3 large lines text-2xl font-medium text-[#8D99AE]:

"Tired of showing your collection in Stories that disappear in 24 hours?"

"Tired of being questioned about whether you actually own it?"

"Tired of not knowing the 200 other people who care as much as you do?"

Then: border-t-2 border-[#E63946] w-24 my-8

Then: "Rebl is the place you have been looking for." text-3xl text-[#F1FAEE] italic

HOW IT WORKS (py-24):

"Three steps." heading text-4xl font-bold

3 numbered steps in flex row (col on mobile): 1. Buy a drop on Rebl 2. AI story auto-generated 3. Verified + community

POST-PURCHASE SECTION (py-24 dark):

"The moment after the drop is where Rebl lives." heading

"Every platform abandons you the second you click Buy. Rebl starts there." subtext

3 feature cards: AI Story Engine | Owner Rooms | Collector Score

FOR BRANDS (py-24 bg-[#16162A]):

"Know your biggest fans after the drop." heading

3 benefits: Post-purchase data | Verified communities | Tiered loyalty

"Partner with Rebl" red CTA button

COMING SOON (py-24):

"The future is being built." heading

3 cards with gold "COMING SOON" badge:

NFC Authentication | Blockchain Provenance | P2P Resale Market

FOOTER: "Rebl" + "One Profile. Your Stories. Infinite Possibilities."

=== PAGE: src/pages/AuthPage.jsx ===

Props: mode ('login' or 'signup'). Full-screen dark. Centered card max-w-md mt-24.

"Rebl" large bold centered. Subtitle based on mode.

Login: email + password fields.

Signup: email + password + username (lowercase no spaces) + display_name fields.

On signup: supabase.auth.signUp then insert profiles

{id,username,display_name,role:'collector'} then redirect /dashboard

On login: supabase.auth.signInWithPassword then fetch profile.role then redirect /brand-dashboard if brand else /dashboard

Loading spinner on button. Errors via toast. Toggle link between modes.

=== PAGE: src/pages/AddItem.jsx ===

Protected. 3-step wizard. Progress bar top (1/3, 2/3, 3/3).

STEP 1 - ITEM DETAILS:

Fields: Item Name (required) | Brand (required) | Edition | Serial Number

Category select:

sneakers|streetwear|luxury_fashion|watches|art|electronics|concert_tickets|trading_cards|vinyl|other

Purchase Date | Purchase Price INR | Photo (file input, show preview on select)

VIBE TAGS below: "What does this piece say about you? (max 5)"

Clickable chips (toggle on/off, red when selected, max 5):

Anti-mainstream | Grail Hunt | First Love | Trophy Piece | Daily Beater | Investment | Gift | Memory | Community Pick | Future Classic | Statement Piece | Heritage | Underrated | Collab Alert | One of a Kind

Button: "Generate My Story" red full-width

STEP 2 - AI STORY ENGINE:

Loading screen with cycling text (800ms each): "Researching your item..." -> "Crafting your story..." -> "Almost ready..."

Call callAI() from gemini.js:

systemPrompt: "You are the story engine for Rebl, a premium platform for serious collectors in India. Write compelling authentic editorial provenance stories. Tone: passionate, collector-to-collector, never corporate."

prompt: "Collector added: Name:[name] Brand:[brand] Edition:[edition] Category:[category] Date:[date]. Write 160-word provenance story: 1) Cultural significance of item/brand 2-3 sentences 2) What makes this edition special 2-3 sentences 3) Personal prompt starting 'Your story with this piece begins'. Second person. No headers. Just the story."

Show AI story in card: dark bg border-l-4 border-[#E63946] p-6 italic text-[#F1FAEE]

Editable textarea below pre-filled with AI story. Label: "Make it yours"

Buttons: "Save to Collection" red + "Regenerate" ghost

STEP 3 - SAVE:

Upload image to Supabase storage bucket 'item-images', path: {userId}/{Date.now()}.jpg, get public URL

Insert into items:

```
{owner_id,name,brand,edition,serial_number,purchase_date,purchase_price,image_url,ai_story,user_story:editedText,category,vibe_tags}
```

Toast: "Added to your collection!" Redirect /profile/{username}

=== PAGE: src/pages/CollectorProfile.jsx ===

Route /profile/:username. Fetch profile where username=param then items where owner_id=profile.id. isOwn check.

HEADER:

80px avatar circle (initials fallback, dark bg red text). Display name text-2xl bold. @username muted.

Bio if set. Stats row: X Items | X Verified | Score: X | PRO gold badge if is_pro.

Archetype badge chip if collector_archetype set.

isOwn: "Edit Profile" ghost + "Add Item" red button.

COLLECTION GRID: grid-cols-2 md:grid-cols-3 gap-4 mt-8

Item card: aspect-square overflow-hidden rounded-lg relative dark bg.

Image object-cover fills card. Bottom gradient overlay with name+brand text.

Green checkmark badge top-right if is_verified. Vibe tag chips below (max 2 + "+X more").

Click card -> Item Detail bottom sheet modal.

ITEM DETAIL MODAL (bottom sheet, fixed bottom-0 w-full max-h-[85vh] overflow-y-auto dark bg rounded-t-2xl):

Image w-full h-64 object-cover. Name text-xl bold + brand + edition + serial.

Verified badge if is_verified.

BRAND STORY section (only if product_story exists for this item's drop - check product_stories table by drop_id on item's origin drop):

"THE BRAND STORY" text-xs muted tracking-widest label

headline text-lg bold, origin_story text, limited_edition_reason italic

COLLECTOR STORY section:

"YOUR STORY" label. user_story if exists else ai_story.

isOwn + no user_story: "Edit Your Story" ghost button -> textarea to update

Vibe tags row. Date + price. "Join Owner Room" red button (href="#"). X close button top-right.

COLLECTOR DNA section (isOwn + items.length >= 3):

"My Collector DNA" heading. "Generate My DNA" gold-border ghost button.

On click: callAIJSON() system:"Analyse collector personalities from what they own. Be insightful."

prompt:"Items: [name brand category vibe_tags for each]. Return JSON: {archetype:'The Grail Hunter|The Heritage Keeper|The Trend Prophet|The Anti-Mainstream|The Investment Collector|The Storyteller|The Community Builder|The Purist', description:'2 sentences second person', signature_phrase:'one punchy sentence'}"

Show result: archetype large bold + description + italic quote. Save to profiles {collector_archetype,archetype_description,signature_phrase}.

EMPTY STATE: "Your collection starts here." + "Add First Item" red button

=== PAGE: src/pages/PostPurchase.jsx ===

Route /post-purchase/:purchaseId. Fetch purchase, join drop and item.

3 screens with smooth CSS transition slides. Progress dots top.

SCREEN 1 (auto-advance 3000ms):

Full dark screen centered. Fire canvas-confetti on mount:

confetti({particleCount:120,spread:80,colors:['#E63946','#F1FAEE','#FFB703']})

Item name fades in text-5xl bold centered (CSS opacity 0->1 1s). Subtitle after 1s delay: "It's yours. Now let's tell your story." muted.

SCREEN 2:

"Your Story Is Being Written" header. Spinner 2s then call callAI() same prompt as AddItem with item details.

Story in quote card (dark bg border-l-4 red italic p-6).

Textarea below pre-filled. "Save My Story" red -> update items.user_story, update purchases set story_completed=true, advance screen 3.

SCREEN 3:

"You're Not Alone" header.

Count: query items where lower(name) like lower(item.name), show "X other collectors own this piece"

3 action cards stacked full-width gap-4:

Red bg card: Join Owner Room -> Discord placeholder href="#" -> set purchases.community_joined=true

Dark red-border card: Add to Your Profile -> toast "Added to your vault"

Dark red-border card: Share Your Story -> navigator.clipboard.writeText(window.location.href) -> toast "Link copied"

"View My Profile" ghost button -> /profile/{username}

On completion: update profiles set collector_score=collector_score+10 where id=user.id

=== PAGE: src/pages/Dashboard.jsx ===

Protected. Fetch profile + items + upcoming drops. Sticky dark navbar: "Rebl" left + "Add Item" red button right + avatar.

Sections:

"Welcome back, [display_name]" + Collector Score badge red.

4 quick stat cards: total items | verified items | items with user_story | purchases where community_joined.

MY COLLECTION: horizontal scroll flex overflow-x-auto gap-4. First card "+" dashed border -> /add-item. Then last 5 items mini cards. "View Full Collection" link -> /profile/{username}.

UPCOMING DROPS: "Drops You'll Want". Fetch 3 from drops where status in (upcoming, live) order drop_date. Each card: image, brand, name, price, status badge, "Check Drop" button -> /brand/{brand_slug}.

STORIES TO COMPLETE: if items with no user_story exist: banner "X items without a story" + list with "Write Story" button (opens inline textarea to update user_story).

REBL PRO (if not is_pro): gold-border card "Unlock Rebl Pro - 999/year". Benefits list. "Upgrade" button -> toast "Coming soon! Email hello@rebl.in"

=== PAGE: src/pages/Vault.jsx ===

Route /vault/:username. Fetch profile + items. isOwn.

Header: "[DisplayName]'s Vault" text-3xl bold. Archetype badge. Stats: Items | Verified | Brands | Est Value sum(purchase_price). isOwn: "Add Item" red button.

4 view tabs (Wall | By Brand | By Category | Timeline). Active = red underline.

WALL: CSS columns-2 md:columns-3 gap-4. Each item: break-inside-avoid mb-4, image rounded-lg w-full, name+brand overlay. Click -> Item Detail modal.

BY BRAND: group items by brand using reduce(). Each group: brand heading + horizontal scroll of item cards.

BY CATEGORY: same but grouped by category.

TIMELINE: flex-col gap-8. Each item sorted by created_at desc: date left in Space Mono muted + item card right + first 100 chars story. CSS left border connecting entries.

=== PAGE: src/pages/BrandPage.jsx ===

Route /brand/:slug. Fetch brand + drops + product_stories.

Brand header: logo + name text-3xl bold + description + tier badge chip.

ACTIVE DROPS grid: image, name, price, units, status badge (green=live red=sold_out gold=upcoming), countdown timer, "Claim Yours" red button.

UPCOMING: grid with countdown + "Notify Me" ghost button (toast "We will let you know!").

COMMUNITY: "X collectors own [Brand] pieces on Rebl" + 5 collector avatars fetched from purchases+profiles.

PRODUCT STORY: if published story, show editorial section with headline large + origin + why_limited italic.

=== PAGE: src/pages/BrandSignup.jsx ===

Route /brand/signup. 3-step wizard with progress indicator.

STEP 1: Brand name | Slug (auto from name, editable) | Preview "Your drop page: {slug}.rebl.in" | Email | Password | Description 140 chars | Logo upload circle preview.

STEP 2: "What does your brand create?"

Grid 3-col desktop 2-col mobile. Each category card: emoji + label. Click=select red border. Click selected=set PRIMARY gold star. One primary required.

Categories: sneakers(Sneakers) | streetwear(Streetwear) | luxury_fashion(Luxury Fashion) | watches(Watches) | art(Art) | electronics(Electronics) | concert_tickets(Concerts/Events) | sports_memorabilia(Sports) | trading_cards(Trading Cards) | vinyl_music(Vinyl) | books_rare(Rare Books) | other(Other)

STEP 3: "How do you create exclusivity?" Multi-select checkboxes:

Numbered/Serialised Editions | Time-Limited Drops | Quantity-Limited | Membership/Tier Access | Collab Drops | Seasonal Releases | Experience Drops

"Launch My Brand" red full-width button.

ON SUBMIT:

1. supabase.auth.signUp

2. insert brands

{name,slug,description,tier:'starter',limited_edition_strategies:selectedStrategies}

3. insert brand_categories for each selected category (is_primary=true for primary)

4. insert brand_subdomains {brand_id,subdomain:slug}

5. insert customer_tiers x3: {name:'Follower',tier_level:1,color:'888888'}

{name:'Insider',tier_level:2,color:'C0A060',min_purchases:2}

{name:'Legend',tier_level:3,color:'FFD700',min_purchases:5}

6. insert profiles {id,username:slug,display_name:brandName,role:'brand'}

7. redirect /brand-dashboard with toast "Brand launched!"

=== PAGE: src/pages/BrandDashboard.jsx ===

Protected brand role. Fetch brand + primary brand_category + customer_tiers + drops + purchases.

CATEGORY LANGUAGE object for label adaptation:

```
const LANG = {
```

```
  sneakers:{drop:'Drop',product:'Pair',community:'Sneakerheads'},
```

```

streetwear:{drop:'Release',product:'Piece',community:'Crew'},
luxury_fashion:{drop:'Edition',product:'Piece',community:'Collectors'},
watches:{drop:'Release',product:'Timepiece',community:'Enthusiasts'},
art:{drop:'Launch',product:'Work',community:'Collectors'},
electronics:{drop:'Drop',product:'Unit',community:'Community'},
concert_tickets:{drop:'Show',product:'Ticket',community:'Fans'},
other:{drop:'Drop',product:'Item',community:'Community'},
}

```

Use LANG[primaryCategory] throughout. Default to LANG.other if not found.

Layout: fixed left sidebar w-60 on desktop (hidden on mobile), bottom tab bar on mobile.

Sidebar: brand logo + name + "[slug].rebl.in" link clickable -> /s/{slug}.

SIDEBAR/BOTTOM TABS: Overview | {lang.drop}s | Story Builder | Customers | Campaigns | Settings

TAB OVERVIEW:

"Welcome back, [brandName]". Subdomain pill: "[slug].rebl.in ->" clickable.

4 stat cards: Units Sold (count purchases) | Active Community (count unique buyers 90 days) | Story Rate % (story_completed=true / total purchases * 100) | Community Rate % (community_joined=true / total * 100).

POST-PURCHASE FUNNEL using recharts BarChart horizontal:

```

data: [{stage:'Buyers',value:totalBuyers},{stage:'Story
Done',value:storyCount},{stage:'Community',value:communityCount}]

```

Red bars. Label "Post-Purchase Journey".

Recent Activity: last 10 purchases with username + item name + timestamp.

TAB DROPS:

Header "Your {lang.drop}s" + "Create New {lang.drop}" red button -> opens modal.

Table: Name | Status badge | Units Total | Units Sold | Story Rate% | Date | Edit button.

CREATE MODAL fields: Name | Description | Image upload | Price INR | Drop date | Status select (upcoming/live/sold_out) | Total Units | Access (Public/Tier-gated) | Category-specific extras.

Save -> insert drops.

TAB STORY BUILDER:

Select drop from dropdown -> load/create product_story for that drop.

Form fields: Headline 80 chars | Origin Story 300 chars | Design Intent 300 chars | Why Limited 200 chars | Behind the Scenes optional.

"Generate with AI" button: call AIJSON() system: "Write authentic editorial product stories for collector brands. Never corporate."

prompt: "Brand:[name] Category:[category] Drop:[dropName] Description:[desc] Strategies:[strategies]. JSON: {headline:'12 words max punchy',origin:'2-3 sentences brand DNA',design_intent:'2-3 sentences what expressed',why_limited:'1-2 sentences honest scarcity'}"

Parse and fill fields. "Save and Publish" -> upsert product_stories published=true.

TAB CUSTOMERS:

SECTION A - TIER CONFIG: 3 tier cards showing name+color badge+count+thresholds. Edit modal per tier: name|color picker|perks textarea one per line|thresholds (min_spend,min_purchases,min_collector_score,min_story_completions)|logic toggle ANY/ALL|backstage toggle. Save to customer_tiers.

SECTION B - COLLECTOR LIST: fetch collector_tier_assignments joined profiles for this brand. Table: avatar|username|tier badge (colored dot+name)|total_spend|purchase_count|collector_score|last active (created_at). Filter by tier dropdown. Sort by any column header click.

SMART CONTACT QUEUE:

"Who to reach out to today" heading.

Query purchases for this brand:

story_completed=false AND created_at < now()-interval 7 days -> yellow chip "Story not written"

community_joined=false AND created_at < now()-interval 14 days -> red chip "Community not joined"

EXTRACT(DOY from created_at) = EXTRACT(DOY from now()) AND EXTRACT(YEAR from created_at) < EXTRACT(YEAR from now()) -> green chip "Drop anniversary"

Each row: avatar | username | item name | trigger chip | "Send" button.

Send -> modal with AI-generated message: callAI() with trigger+item+brand context.

Editable. Send -> insert post_purchase_contacts.

TAB CAMPAIGNS:

7 template cards 2-col grid:

1 Drop Announcement | 2 Early Access Invite | 3 Backstage Invite | 4 Story Nudge | 5 Loyalty Reward | 6 Re-engagement | 7 Custom Message

Click template -> Campaign Composer modal:

Campaign name | Target tiers multi-select + "X collectors" count | Subject editable | Message body textarea.

"Improve with AI" -> callAI() rewrites message in brand voice.

"Send Now" red -> insert campaigns {status:'sent',sent_at:now()} + toast "Sent to X collectors!"

Campaign History table: Name | Type | Recipients | Date | Status.

TAB SETTINGS:

Brand profile: name, description, logo upload. Save to brands.

Subdomain: read-only slug + copy button. Hero headline + subheadline. Theme primary + accent color pickers. Save to brand_subdomains.

"Preview My Page" button -> window.open('/s/'+slug).

Pricing card: Starter:Free | Growth:15000/mo | Enterprise:75000/mo. "Upgrade" -> toast "Email hello@rebl.in".

=== PAGE: src/pages/BrandSubdomainPage.jsx ===

Route /s/:brandSlug. Fetch brand + brand_subdomains + drops + product_stories.

Apply theme: inject CSS vars from brand_subdomains onto root div style.

Only Rebl branding: small "Powered by Rebl" footer badge.

SECTIONS:

HERO: brand logo centered + hero_headline large + hero_subheadline + "See What's Dropping" scroll CTA using page accent color.

DROPS heading based on category (sneakers:"The Drop" concerts:"The Show" art:"The Collection" default:"Now Available"):

Live drops: full-width card image-left-60% details-right-40%. Price + units remaining + "Claim Yours" button.

Upcoming: grid cards with countdown timers + "Notify Me" button (toast only).

PRODUCT STORY (if published): editorial full-bleed section. Headline large + origin paragraph + why_limited italic.

COMMUNITY: "X collectors own [Brand] pieces on Rebl" + 5 avatars -> link to /brand/{slug}.

FOOTER: "Powered by Rebl" small link.

=== PAGE: src/pages/TribePage.jsx ===

Route /tribe. Protected.

"Find Your People" header. "Real collectors. Verified owners. Same taste as you." sub.

SECTION 1 - SHARED BRANDS: find other collectors who own items from same brands as logged-in user. Horizontal scroll of collector mini-cards: avatar+username+"X shared brands"+"View" button.

SECTION 2 - BY VIBE: all unique vibe_tags from items table. Horizontal chip scroll. Click tag -> filter to collectors with that tag.

SECTION 3 - BY ARCHETYPE: 8 archetype chip cards. Click -> collectors with that archetype.

SECTION 4 - YOUR CITY: if profile.city set, show collectors from same city. Else "Add your city in profile settings" CTA.

=== BUILD ORDER ===

1. Install all packages and setup Tailwind
2. Create .env and both lib files (supabase.js and gemini.js)
3. App.jsx with all routes
4. LandingPage
5. AuthPage
6. AddItem (critical - Gemini AI here)
7. CollectorProfile with DNA feature
8. PostPurchase
9. Dashboard
10. Vault
11. BrandPage
12. BrandSignup
13. BrandDashboard (all tabs)
14. BrandSubdomainPage
15. TribePage

Build all pages completely. Do not use TODO comments or placeholder functions - implement everything. Use consistent dark design system. Mobile-first responsive. When finished run dev server.

=== SUPABASE SQL (DO NOT BUILD IN REACT - RUN THIS SEPARATELY IN SUPABASE SQL EDITOR) ===

```
create table public.profiles (id uuid references auth.users on delete cascade primary key,
username text unique not null, display_name text, bio text, avatar_url text, city text, role
text default 'collector' check (role in ('collector','brand','admin')), collector_score integer
default 0, is_pro boolean default false, collector_archetype text, archetype_description text,
signature_phrase text, created_at timestampz default now());
```

```
create table public.items (id uuid default gen_random_uuid() primary key, owner_id uuid references public.profiles(id) on delete cascade, name text not null, brand text not null, edition text, serial_number text, purchase_date date, purchase_price numeric, image_url text, ai_story text, user_story text, is_verified boolean default false, is_for_sale boolean default false, category text default 'sneakers', vibe_tags text[] default '{}', estimated_value numeric, created_at timestamptz default now());
```

```
create table public.brands (id uuid default gen_random_uuid() primary key, name text not null, slug text unique not null, logo_url text, description text, tier text default 'starter', limited_edition_strategies jsonb default '[]', created_at timestamptz default now());
```

```
create table public.brand_categories (id uuid default gen_random_uuid() primary key, brand_id uuid references public.brands(id) on delete cascade, category text not null, is_primary boolean default false);
```

```
create table public.drops (id uuid default gen_random_uuid() primary key, brand_id uuid references public.brands(id), name text not null, description text, image_url text, total_units integer, price numeric, drop_date timestamptz, status text default 'upcoming', access_type text default 'public', extra_fields jsonb default '{}', created_at timestamptz default now());
```

```
create table public.purchases (id uuid default gen_random_uuid() primary key, user_id uuid references public.profiles(id), drop_id uuid references public.drops(id), item_id uuid references public.items(id), purchase_price numeric, story_completed boolean default false, community_joined boolean default false, created_at timestamptz default now());
```

```
create table public.product_stories (id uuid default gen_random_uuid() primary key, drop_id uuid references public.drops(id) on delete cascade, brand_id uuid references public.brands(id), headline text, origin_story text, design_intent text, limited_edition_reason text, behind_scenes text, published boolean default false);
```

```
create table public.customer_tiers (id uuid default gen_random_uuid() primary key, brand_id uuid references public.brands(id) on delete cascade, name text not null, tier_level integer not null, color text default 'CCCCC', min_spend numeric default 0, min_purchases integer default 0, min_collector_score integer default 0, min_story_completions integer default 0, require_all boolean default false, perks jsonb default '[]', has_backstage_access boolean default false);
```

```
create table public.collector_tier_assignments (id uuid default gen_random_uuid() primary key, collector_id uuid references public.profiles(id), brand_id uuid references public.brands(id), tier_id uuid references public.customer_tiers(id), total_spend numeric default 0, purchase_count integer default 0, unique(collector_id, brand_id));
```

```
create table public.campaigns (id uuid default gen_random_uuid() primary key, brand_id uuid references public.brands(id), name text not null, type text, target_tier_ids uuid[],
```

message_subject text, message_body text, status text default 'draft', sent_at timestampz, open_count integer default 0);

create table public.post_purchase_contacts (id uuid default gen_random_uuid() primary key, brand_id uuid references public.brands(id), purchase_id uuid references public.purchases(id), collector_id uuid references public.profiles(id), trigger text, message text, sent_at timestampz, opened boolean default false);

create table public.brand_subdomains (id uuid default gen_random_uuid() primary key, brand_id uuid references public.brands(id) unique, subdomain text unique not null, custom_domain text, theme_primary text default '0F0F1A', theme_accent text default 'E63946', hero_headline text, hero_subheadline text);

alter table public.profiles enable row level security;

alter table public.items enable row level security;

alter table public.brands enable row level security;

alter table public.drops enable row level security;

alter table public.purchases enable row level security;

alter table public.product_stories enable row level security;

alter table public.customer_tiers enable row level security;

alter table public.collector_tier_assignments enable row level security;

alter table public.campaigns enable row level security;

alter table public.post_purchase_contacts enable row level security;

alter table public.brand_subdomains enable row level security;

create policy "p_read" on profiles for select using (true);

create policy "p_edit" on profiles for update using (auth.uid() = id);

create policy "p_insert" on profiles for insert with check (auth.uid() = id);

create policy "i_read" on items for select using (true);

create policy "i_manage" on items for all using (auth.uid() = owner_id);

create policy "b_read" on brands for select using (true);

create policy "b_insert" on brands for insert with check (true);

create policy "d_read" on drops for select using (true);

create policy "s_read" on product_stories for select using (true);

```

create policy "t_read" on customer_tiers for select using (true);

create policy "sub_read" on brand_subdomains for select using (true);

create policy "sub_insert" on brand_subdomains for insert with check (true);

insert into storage.buckets (id,name,public) values ('item-images','item-
images',true),('brand-logos','brand-logos',true),('drop-images','drop-images',true);

create policy "stor_read" on storage.objects for select using (bucket_id in ('item-
images','brand-logos','drop-images'));

create policy "stor_write" on storage.objects for insert with check
(auth.role()='authenticated');

insert into public.brands (name,slug,description,tier) values
('VegNonVeg','vegnongveg','India premier sneaker
destination.','growth'),('Superkicks','superkicks','Premium sneakers for India collector
community.','starter');

insert into public.brand_subdomains (brand_id,subdomain) select id,slug from
public.brands;

insert into public.drops (brand_id,name,description,total_units,price,drop_date,status)
select b.id,'Air Jordan 1 High Chicago','The shoe that started it all. 1000
pairs.',1000,12999,now()+interval '7 days','upcoming' from public.brands b where
b.slug='vegnongveg';

insert into public.drops (brand_id,name,description,total_units,price,drop_date,status)
select b.id,'Nike Dunk Low Panda','Black and white. 500 pairs only.',500,8999,now()-interval
'30 minutes','live' from public.brands b where b.slug='superkicks';

```